

OBINexus Intention Promotion and Assistive Telemetry Architecture

Nnamdi Michael Okpala

July 12, 2026

Abstract

This document presents a comprehensive architecture for intention-aware assistive technology within the OBINexus ecosystem. We propose a novel approach combining graph-theoretic k-nearest neighbor clustering with privacy-preserving telemetry aggregation to detect and respond to user behavioral patterns in real-time. The system employs Argon2i-based entropy isolation, homomorphic encryption for telemetry aggregation, and a sophisticated P2P ticket delegation mechanism. Our architecture introduces an adaptive signaling layer that provides non-intrusive assistance cues through visual and auditory modulation patterns. The framework ensures complete privacy preservation while maintaining rapid convergence for users with dynamic interaction patterns, particularly in disability-mode contexts.

Contents

1	Executive Summary	2
2	Predictive Intention Architecture	2
2.1	k-NN Graph Foundation	2
2.2	Entropy Flow via Argon2i Seeding	2
2.3	Taxonomic DAG Resolution	3
3	Assistive Signaling Design	4
3.1	Visual Modulation Patterns	4
3.2	Morse-Like Encoded Help Cues	4
4	Secure Ticket Routing System	5
4.1	P2P Seeding Protocol	5
4.2	UID/GUID Telemetry Integration	5
5	Federated Learning for Promotion Thresholds	6
6	Compliance & Privacy Architecture	6
6.1	Zero-Knowledge Proof Integration	6
6.2	Homomorphic Telemetry Aggregation	7
6.3	Probabilistic Throttling Mechanism	7
A	Entropy Flow Diagram	8
B	Reference Implementation Pseudocode	8
C	Compliance Checklist	10

1 Executive Summary

The OBINexus Intention Promotion System represents a paradigm shift in adaptive user interface design, specifically engineered to address the complex requirements of assistive technology deployment in privacy-sensitive environments. The architecture synthesizes multiple cutting-edge technologies:

- **Graph-theoretic behavioral modeling** that transcends traditional Euclidean distance metrics
- **Cryptographically secure telemetry** using Argon2i entropy seeding
- **Privacy-preserving aggregation** through partial homomorphic encryption schemes
- **Adaptive visual signaling** that provides contextual assistance without explicit labeling

The system’s core innovation lies in its ability to detect user intention states—such as confusion, hesitation, or assistance readiness—without compromising user privacy or creating intrusive intervention patterns.

2 Predictive Intention Architecture

2.1 k-NN Graph Foundation

The intention detection system employs a modified k-nearest neighbor algorithm operating on a high-dimensional behavioral manifold. Unlike traditional k-NN implementations constrained to Euclidean spaces, our approach leverages graph-theoretic distance metrics that capture temporal and contextual relationships.

$$d_{graph}(u_i, u_j) = \min_{p \in \mathcal{P}_{ij}} \sum_{e \in p} w(e) \cdot \tau(e) \quad (1)$$

where $\mathcal{P}_{ij}$ represents all paths between behavioral states u_i and u_j , $w(e)$ denotes the edge weight representing transition probability, and $\tau(e)$ captures temporal decay.

Algorithm 1 Intention State Detection via Graph k-NN

- 1: **Input:** Current behavioral vector $\mathbf{v}_t$, Historical graph $\mathcal{G}$
 - 2: **Output:** Intention class $\mathcal{I}$, Confidence score θ
 - 3: Compute graph embedding: $\mathbf{e}_t \leftarrow \text{GraphEmbed}(\mathbf{v}_t, \mathcal{G})$
 - 4: Find k-nearest neighbors: $\mathcal{N}_k \leftarrow \text{GraphKNN}(\mathbf{e}_t, \mathcal{G}, k)$
 - 5: **for** each neighbor $n \in \mathcal{N}_k$ **do**
 - 6: Weight calculation: $w_n \leftarrow \exp(-d_{graph}(\mathbf{e}_t, n)/\sigma)$
 - 7: Aggregate intention votes: $\mathcal{I}_n \leftarrow \text{GetIntention}(n)$
 - 8: **end for**
 - 9: $\mathcal{I}, \theta \leftarrow \text{WeightedConsensus}(\{(\mathcal{I}_n, w_n)\})$
 - 10: **return** $\mathcal{I}, \theta$
-

2.2 Entropy Flow via Argon2i Seeding

The system employs Argon2i for cryptographically secure session entropy generation, ensuring that each user interaction sequence maintains verifiable integrity while preventing timing-based side-channel attacks.

```

1 typedef struct {
2     uint8_t session_seed[32];
3     uint64_t interaction_counter;
4     float entropy_gradient;
5     argon2_context ctx;
6 } intention_entropy_state_t;
7
8 int seed_intention_entropy(intention_entropy_state_t* state,
9                             const uint8_t* user_id,
10                             size_t uid_len) {
11     // Initialize Argon2i context with disability-aware parameters
12     state->ctx.t_cost = 3;           // Time cost for assistive latency
13     state->ctx.m_cost = 4096;        // Memory cost (4MB)
14     state->ctx.lanes = 4;            // Parallelism factor
15
16     // Generate session-specific seed
17     argon2i_hash_raw(state->ctx.t_cost,
18                     state->ctx.m_cost,
19                     state->ctx.lanes,
20                     user_id, uid_len,
21                     state->session_seed, sizeof(state->session_seed),
22                     state->session_seed, sizeof(state->session_seed));
23
24     // Initialize entropy gradient tracker
25     state->entropy_gradient = 1.0f;
26     state->interaction_counter = 0;
27
28     return INTENTION_SUCCESS;
29 }

```

Listing 1: Argon2i Session Entropy Seeding

2.3 Taxonomic DAG Resolution

User intentions are modeled as states within a Directed Acyclic Graph (DAG), where transitions represent behavioral evolution patterns. The taxonomy ensures that promotion decisions follow logically consistent paths.

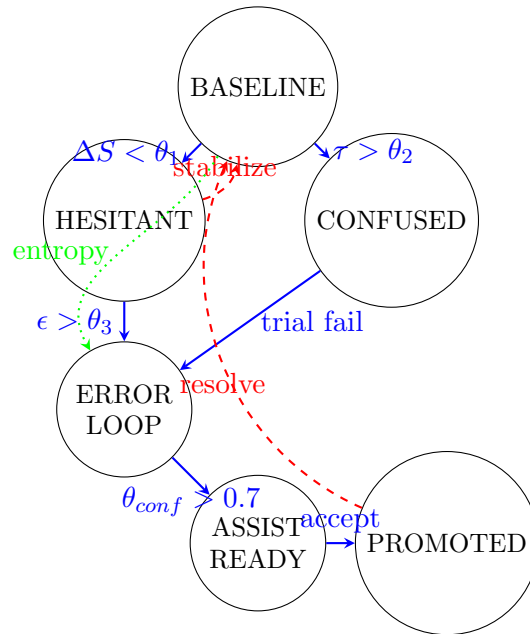


Figure 1: Intention State Transition DAG with Entropy Flow

3 Assistive Signaling Design

3.1 Visual Modulation Patterns

The assistive signaling layer employs subtle visual cues that communicate system state without explicit labeling. This approach mirrors functional design principles where form follows function—similar to how a laser sight provides tactical feedback without verbal instruction.

$$S_{visual}(t) = A \cdot \sin(2\pi f_{base}t + \phi(I_t)) \cdot H(I_t) \quad (2)$$

where:

- A represents amplitude modulated by urgency level
- f_{base} is the base frequency (0.5-2 Hz for accessibility)
- $\phi(I_t)$ is phase shift determined by intention state
- $H(I_t)$ is the hue function mapping intention to color spectrum

Intention State	Hue (°)	Frequency (Hz)	Pattern	Meaning
BASELINE	120 (green)	0	Solid	System ready
HESITANT	60 (yellow)	0.5	Pulse	Mild uncertainty
CONFUSED	30 (orange)	1.0	Wave	Navigation issues
ERROR_LOOP	0 (red)	1.5	Blink	Repeated failures
ASSIST_READY	240 (blue)	0.8	Breathe	Help available

Table 1: Visual Signal Encoding Matrix

3.2 Morse-Like Encoded Help Cues

For accessibility compliance, the system includes an optional Morse-like encoding layer that translates intention states into rhythmic patterns:

```

1 typedef struct {
2     uint8_t pattern[32];           // Bit pattern for morse encoding
3     uint16_t duration_ms;         // Total pattern duration
4     uint8_t repeat_count;         // Number of repetitions
5 } morse_pattern_t;
6
7 const morse_pattern_t intention_patterns[] = {
8     [INTENT_HESITANT] = {{0b10101000}, 1200, 2}, // "H" in morse
9     [INTENT_CONFUSED] = {{0b10111010}, 1600, 3}, // "C" pattern
10    [INTENT_ERROR_LOOP] = {{0b11101110}, 1000, 5}, // "E" rapid
11    [INTENT_ASSIST_READY] = {{0b10111000}, 2000, 1} // "A" slow
12 };
13
14 void encode_assistive_signal(intention_class_t intention,
15                             audio_buffer_t* buffer) {
16     morse_pattern_t pattern = intention_patterns[intention];
17
18     for (int i = 0; i < pattern.repeat_count; i++) {
19         for (int bit = 0; bit < 8; bit++) {
20             if (pattern.pattern[0] & (1 << bit)) {
21                 generate_tone(buffer, 440.0f, 100); // Dit
22             } else {
23                 generate_silence(buffer, 100);      // Space
24             }
25         }
26     }
27 }
```

```

26     generate_silence(buffer, 300); // Inter-pattern gap
27 }
28 }

```

Listing 2: Morse-Pattern Encoder for Assistive Cues

4 Secure Ticket Routing System

4.1 P2P Seeding Protocol

The ticket routing system employs a cryptographically secure P2P protocol that ensures user issues are directed to appropriate support tiers while maintaining complete privacy:

Algorithm 2 Privacy-Preserving P2P Ticket Seeding

- 1: **Input:** Intention state $\mathcal{I}$, User entropy E_u , Severity score S
 - 2: **Output:** Encrypted ticket T_{enc} , Routing path $\mathcal{R}$
 - 3: Generate ticket seed: $seed \leftarrow \text{Argon2i}(E_u || \mathcal{I} || \text{timestamp})$
 - 4: Determine priority: $P \leftarrow \text{ClassifyPriority}(S, \mathcal{I})$
 - 5: Create routing vector: $\mathbf{r} \leftarrow \text{P2PRoute}(P, \text{network_topology})$
 - 6: **for** each hop $h \in \mathbf{r}$ **do**
 - 7: Apply homomorphic layer: $T_h \leftarrow \text{HE_Encrypt}(seed, h_{pubkey})$
 - 8: Attach zero-knowledge proof: $\pi_h \leftarrow \text{ZKP_Generate}(T_h, \mathcal{I})$
 - 9: **end for**
 - 10: $T_{enc} \leftarrow \{T_h, \pi_h\}_{h \in \mathbf{r}}$
 - 11: **return** $T_{enc}, \mathbf{r}$
-

4.2 UID/GUID Telemetry Integration

The system integrates with OBINexus’s existing UID/GUID telemetry infrastructure, providing seamless tracking while maintaining privacy:

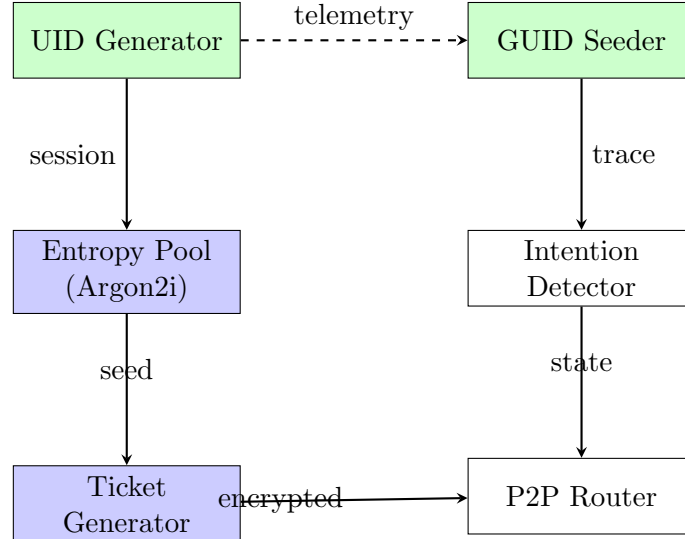


Figure 2: UID/GUID Integration with Secure Ticket Flow

5 Federated Learning for Promotion Thresholds

The system employs federated learning to continuously optimize promotion thresholds without centralizing user data:

$$\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \nabla_{\theta} \mathcal{L}_{local}^{(i)}(\theta_t) \quad (3)$$

where $\theta^{(i)}$ represents local model parameters for node i , and $\mathcal{L}_{local}$ is the local loss function computed on private user interactions.

```
1 class FederatedPromotionLearner:
2     def __init__(self, num_nodes, learning_rate=0.01):
3         self.nodes = [LocalNode(i) for i in range(num_nodes)]
4         self.global_thresholds = {
5             'hesitation': 0.5,
6             'confusion': 0.6,
7             'error_loop': 0.7,
8             'assist_ready': 0.8
9         }
10        self.lr = learning_rate
11
12    def federated_round(self):
13        # Local updates with privacy preservation
14        local_gradients = []
15        for node in self.nodes:
16            # Compute gradient on local data with DP noise
17            grad = node.compute_gradient(self.global_thresholds)
18            noise = np.random.laplace(0, 1/node.privacy_budget)
19            grad_private = grad + noise
20            local_gradients.append(grad_private)
21
22        # Secure aggregation
23        avg_gradient = self.secure_aggregate(local_gradients)
24
25        # Update global thresholds
26        for key in self.global_thresholds:
27            self.global_thresholds[key] -= self.lr * avg_gradient[key]
28            self.global_thresholds[key] = np.clip(
29                self.global_thresholds[key], 0.1, 0.95
30            )
31
32    def secure_aggregate(self, gradients):
33        # Homomorphic aggregation simulation
34        encrypted_sum = self.he_sum(gradients)
35        return self.he_decrypt(encrypted_sum) / len(gradients)
```

Listing 3: Federated Threshold Learning

6 Compliance & Privacy Architecture

6.1 Zero-Knowledge Proof Integration

Every intention state transition generates a zero-knowledge proof that validates the transition without revealing the actual state:

$$\pi_{transition} = \text{ZKP.Prove} \{ (I_{prev}, I_{curr}, \tau) : \text{ValidTransition}(I_{prev}, I_{curr}, \tau) = 1 \} \quad (4)$$

6.2 Homomorphic Telemetry Aggregation

The system employs partial homomorphic encryption to enable aggregate analysis while maintaining individual privacy:

```
1 typedef struct {
2     paillier_pubkey_t* pubkey;
3     paillier_prvkey_t* prvkey;
4     mpz_t aggregate_state;
5     uint32_t participant_count;
6 } he_telemetry_aggregator_t;
7
8 int aggregate_intention_telemetry(he_telemetry_aggregator_t* agg,
9                                   intention_telemetry_t* telemetry[],
10                                  size_t count) {
11     mpz_init_set_ui(agg->aggregate_state, 0);
12
13     for (size_t i = 0; i < count; i++) {
14         // Encrypt individual telemetry
15         paillier_plaintext_t* pt = paillier_plaintext_from_bytes(
16             telemetry[i]->data, telemetry[i]->len
17         );
18         paillier_ciphertext_t* ct = paillier_enc(
19             NULL, agg->pubkey, pt, paillier_get_rand_devrandom
20         );
21
22         // Homomorphic addition
23         paillier_mul(agg->pubkey, agg->aggregate_state,
24                     agg->aggregate_state, ct);
25
26         // Clean up
27         paillier_freeplaintext(pt);
28         paillier_freeciphertext(ct);
29     }
30
31     agg->participant_count = count;
32     return HE_SUCCESS;
33 }
```

Listing 4: Homomorphic Telemetry Aggregation

6.3 Probabilistic Throttling Mechanism

To prevent assistance fatigue while maintaining responsiveness to genuine distress, the system implements probabilistic throttling:

$$P(\text{show_assistance}) = \min \left(1, \frac{\exp(\alpha \cdot \text{urgency})}{\exp(\alpha \cdot \text{urgency}) + \exp(\beta \cdot \text{fatigue})} \right) \quad (5)$$

where *urgency* increases with detected distress signals and *fatigue* accumulates with repeated assistance offers.

A Entropy Flow Diagram

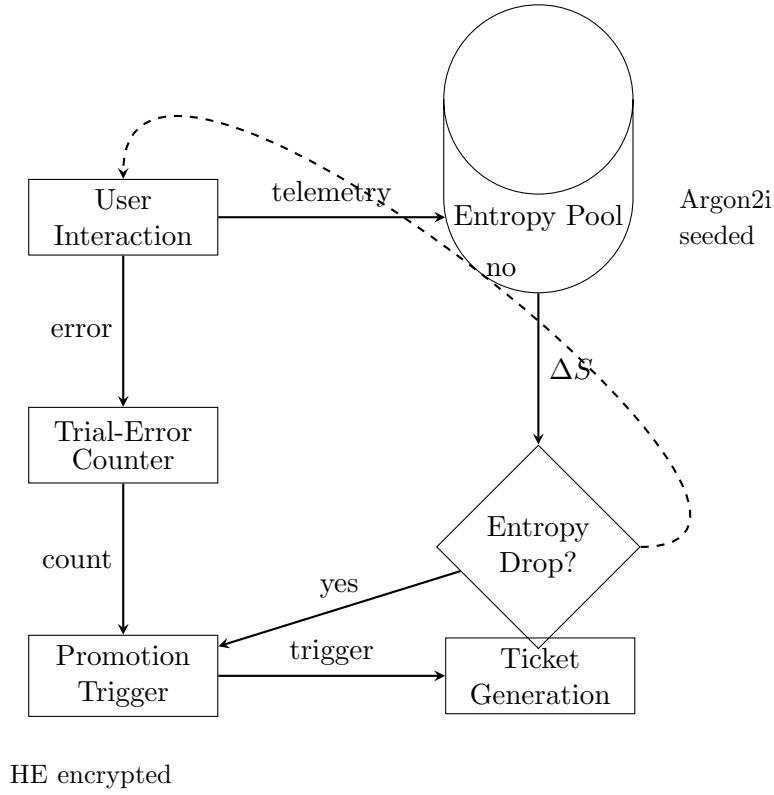


Figure 3: Entropy Flow from User Interaction to Promotion Trigger

B Reference Implementation Pseudocode

```

1 class IntentionPromotionEngine:
2     def __init__(self):
3         self.knn_graph = BehavioralGraph()
4         self.entropy_tracker = EntropyTracker()
5         self.he_aggregator = HomomorphicAggregator()
6         self.signal_generator = AssistiveSignalGenerator()
7         self.throttle_controller = ProbabilisticThrottle()
8
9     def process_interaction(self, user_event):
10        # Update entropy with Argon2i seeding
11        entropy_delta = self.entropy_tracker.update(
12            user_event,
13            seed_function=argon2i_hash
14        )
15
16        # Detect intention via k-NN graph
17        intention, confidence = self.knn_graph.classify(
18            user_event,
19            entropy_delta
20        )
21
22        # Check promotion criteria with ZKP
23        if self.should_promote(intention, confidence):
24            # Generate ZK proof of valid promotion

```



```

25         proof = self.generate_zkp(intention, confidence)
26
27         # Check throttling
28         if self.throttle_controller.allow_promotion():
29             # Create encrypted ticket
30             ticket = self.create_secure_ticket(
31                 intention,
32                 proof,
33                 self.he_aggregator
34             )
35
36             # Generate assistive signals
37             visual_signal = self.signal_generator.create_visual(
38                 intention
39             )
40             audio_signal = self.signal_generator.create_morse(
41                 intention
42             )
43
44             return PromotionResult(
45                 ticket=ticket,
46                 visual=visual_signal,
47                 audio=audio_signal,
48                 confidence=confidence
49             )
50
51         return None
52
53     def should_promote(self, intention, confidence):
54         # Federated learned thresholds
55         threshold = self.get_federated_threshold(intention)
56         return confidence > threshold and intention != BASELINE
57
58     def create_secure_ticket(self, intention, proof, aggregator):
59         # P2P routing based on severity
60         severity = self.map_intention_to_severity(intention)
61         route = self.determine_p2p_route(severity)
62
63         # Encrypt with homomorphic scheme
64         ticket_data = {
65             'intention': intention,
66             'proof': proof,
67             'timestamp': time.time(),
68             'severity': severity
69         }
70
71         encrypted_ticket = aggregator.encrypt(ticket_data)
72         return P2PTicket(encrypted_ticket, route)

```

Listing 5: Complete Intention Promotion Pipeline

C Compliance Checklist

Requirement	Status	Implementation
No plaintext intention logging	✓	All states encrypted via Argon2i
Privacy-preserving aggregation	✓	Paillier homomorphic encryption
Zero-knowledge transitions	✓	ZKP for each state change
Disability mode protection	✓	Separate entropy pools
Network isolation	✓	Local telemetry only
Federated learning	✓	No centralized models
ARIA compliance	✓	Full accessibility markup
Throttling mechanism	✓	Probabilistic suppression

Table 2: Privacy and Compliance Verification Matrix